Fundamental Programming in JAVA

PREDEFINED CLASSES & USER DEFINED METHODS

Objectives

- Solve mathematics problems by using the methods in the Math class.
- Represent strings using the String objects.
- Perform operations on String objects such as trimming ,concatenating, comparing, replacing strings etc.
- Define methods with formal parameters.
- Invoke methods with actual parameters.
- Define methods with a return value.
- Define methods without a return value.
- Pass arguments by value.

The String Class

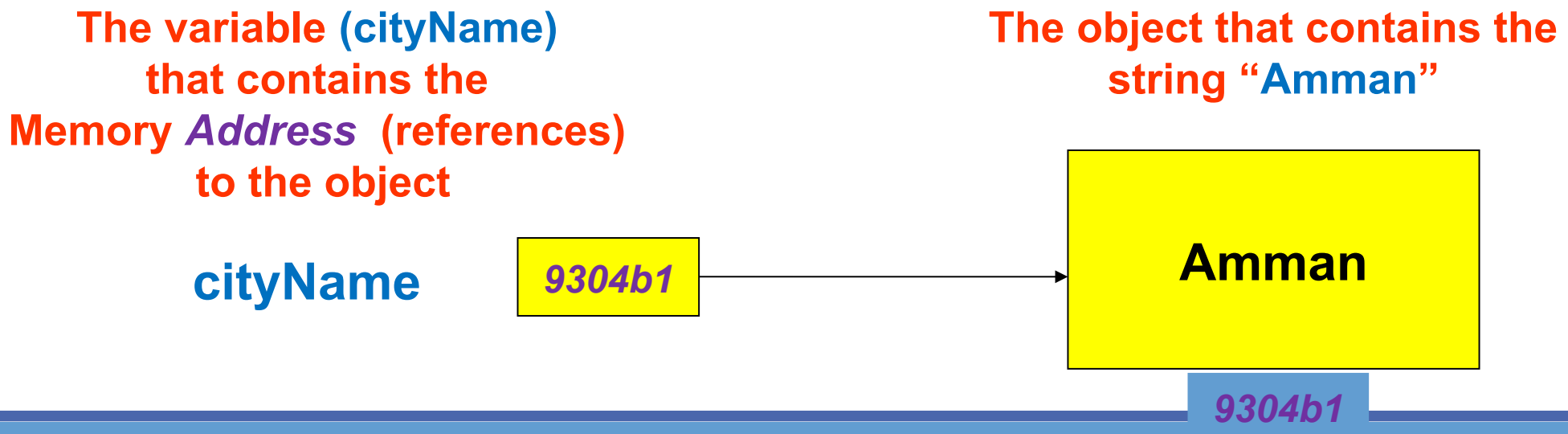
- Java has no primitive data type that holds a series of characters.
- The String class from the Java standard library is used for this purpose.
- In order to be useful, the a variable must be created to reference a String object.
  - Eg. **String number;**
- Notice the **S** in ***String*** is upper case.
- By convention, class names should always begin with an upper case character.

Primitive vs. Reference Variables

- Primitive variables actually contain the value that they have been assigned.
  - Eg. **int number = 25;**
- The value 25 will be stored in the memory location associated with the variable number.
- Objects are **not stored** in variables, however. Objects are **referenced** by variables.

Primitive vs. Reference Variables

- When a variable references an object, it contains the memory address of the object's location.
- Then it is said that the variable **references** the object.
  - Eg. **String cityName = "Amman";**



String Objects

- A variable can be assigned a String literal.
  String value1 = “Welcome to Java";
- **Strings** are the only objects that can be created in this way.
- A variable can be created using the ***new*** keyword.
  String value2 = new String("Java Programming");
- This is the method that all other objects must use when they are created.

Strings Are Immutable

- A String object is immutable; its contents cannot be changed. Does the following code change the contents of the string?

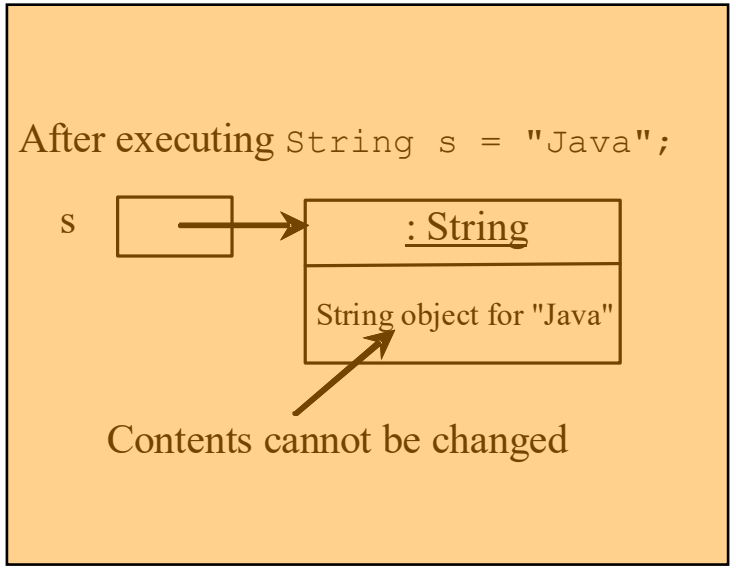
```
String s = "Java";
s = "HTML";
```

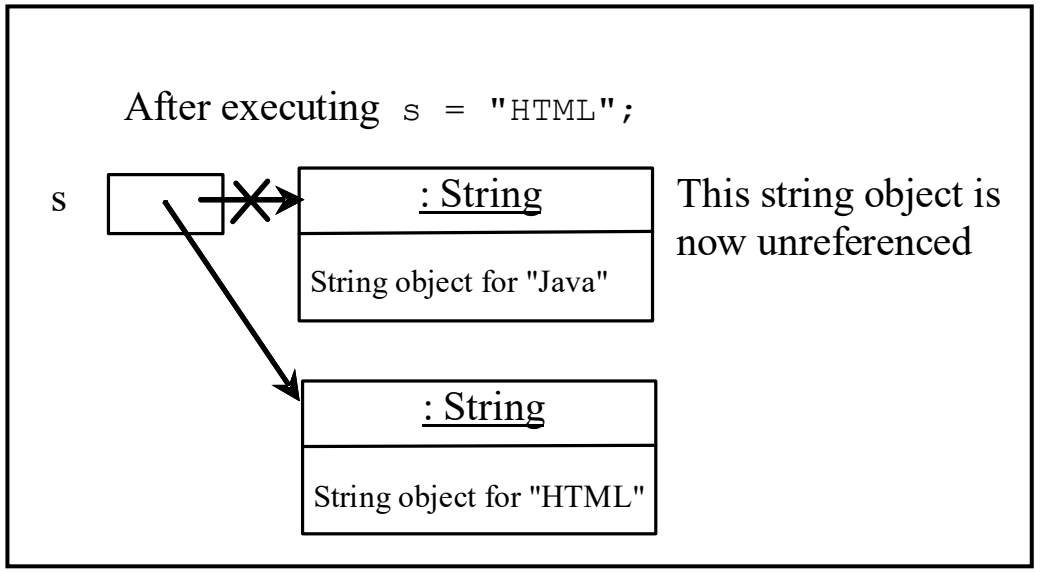
Trace Code (Two Slides)

```
String s = "Java";
```

```
s = "HTML";
```

Code in Orange First



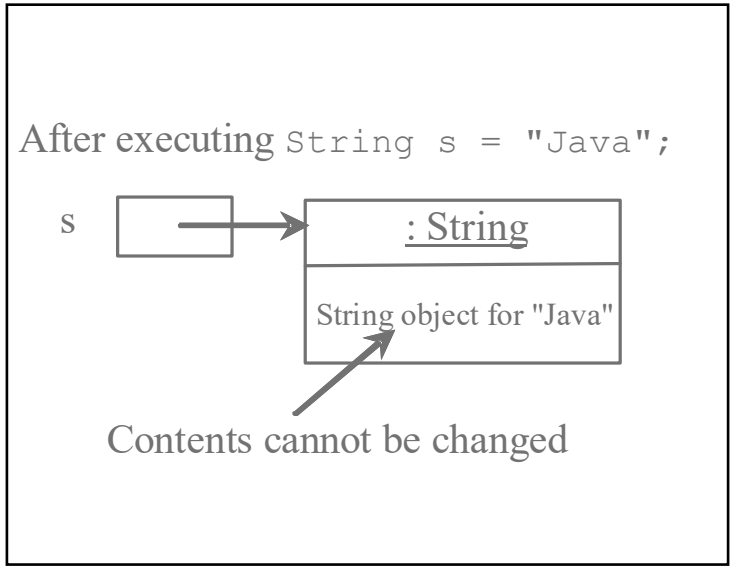


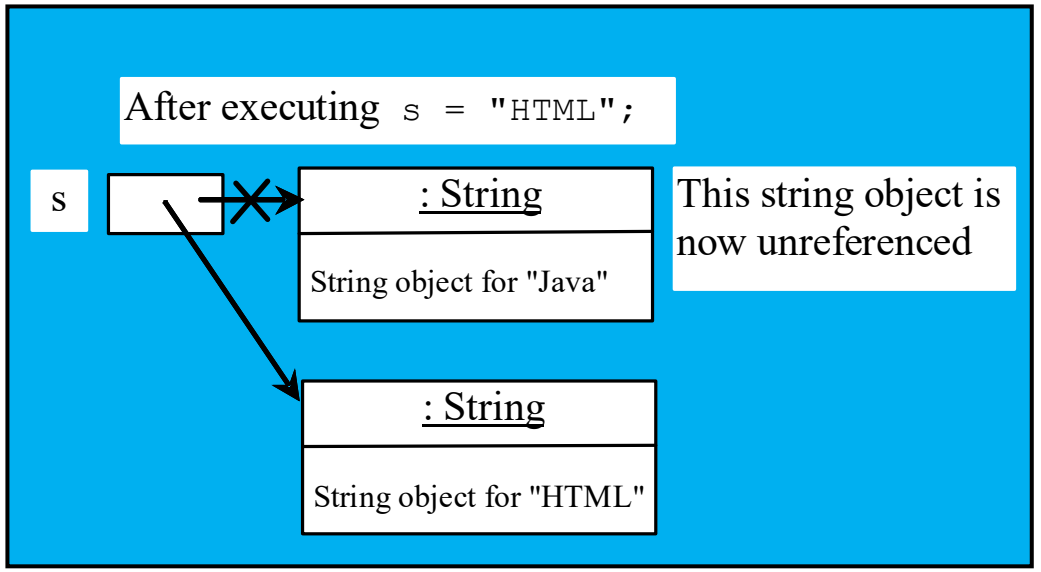
Cont. Trace Code

```
String s = "Java";
```

```
s = "HTML";
```

Code in Blue After





Interned Strings

- Since strings are immutable and are frequently used, to improve efficiency and save memory, the JVM uses a unique instance for string literals with the same character sequence. Such an instance is called interned. For example, the following statements:

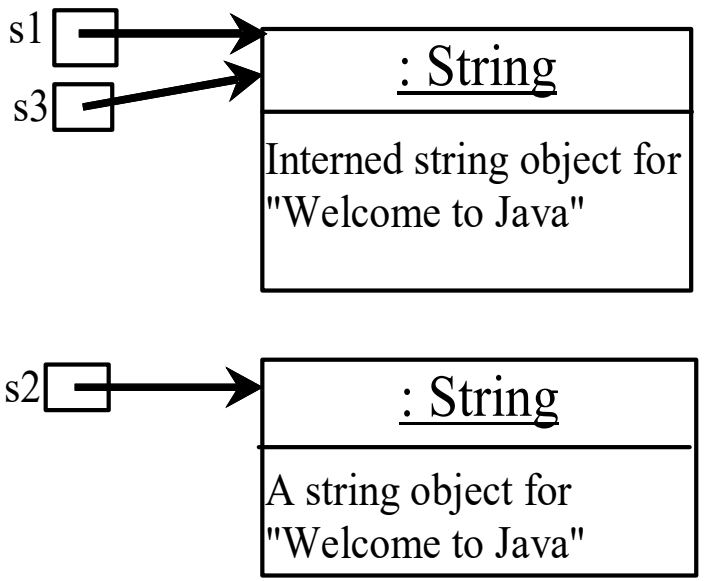
Examples

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";

System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
```



display
s1 == s2 is **false**

s1 == s3 is **true**

A new object is created if you use the new operator.

If you use the string initializer, no new object is created if the interned object is already created.

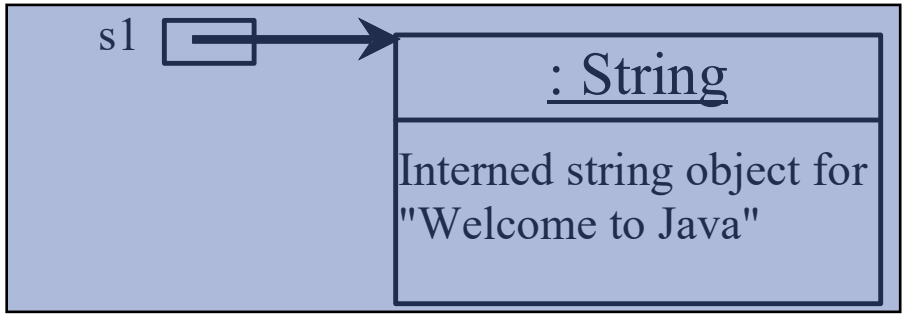
Trace Code (Three Slides)

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```

Code in Blue First



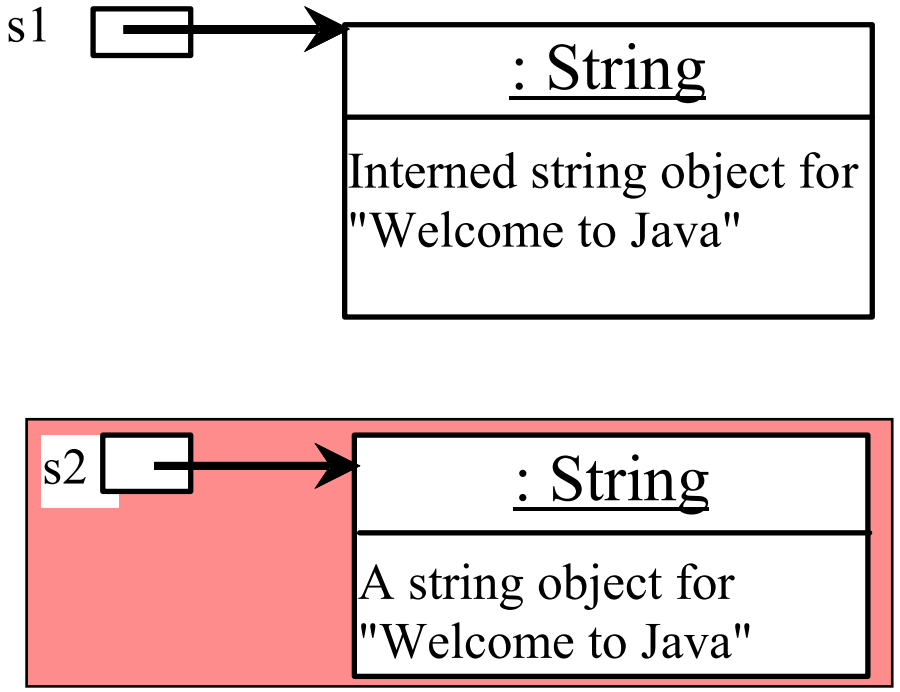
Cont. Trace Code

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```

Code in Red Now



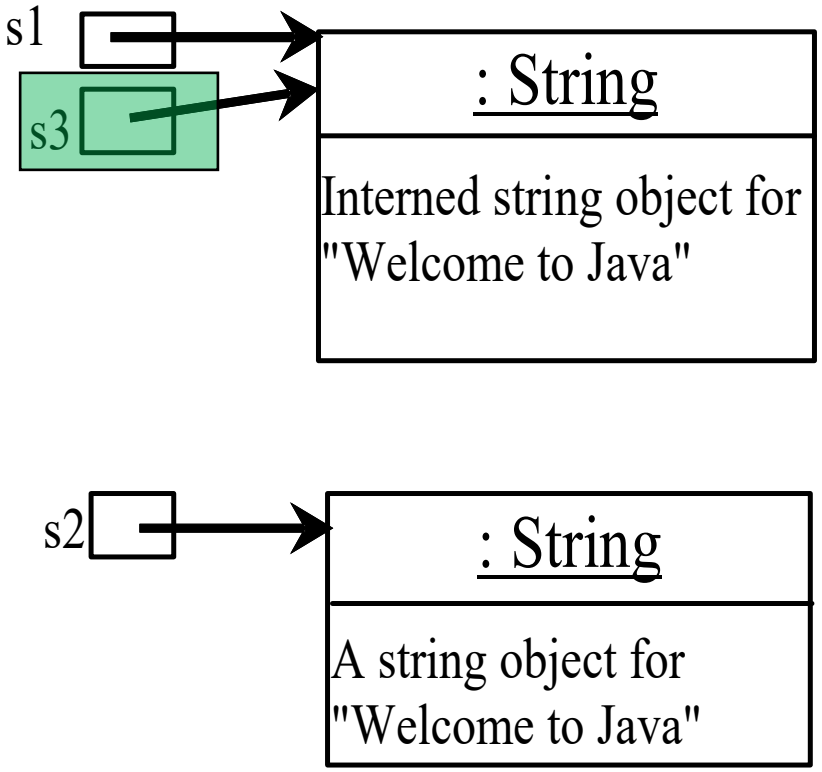
Cont. Trace Code

```
String s1 = "Welcome to Java";

String s2 = new String("Welcome to Java");

String s3 = "Welcome to Java";
```

Code in Green Now



String Methods

- The **String** class contains many methods that help with the manipulation of **String** objects.
- ***String*** objects are **immutable**, meaning that they cannot be changed.
- Many of the methods of a ***String*** object can create new versions of the object.

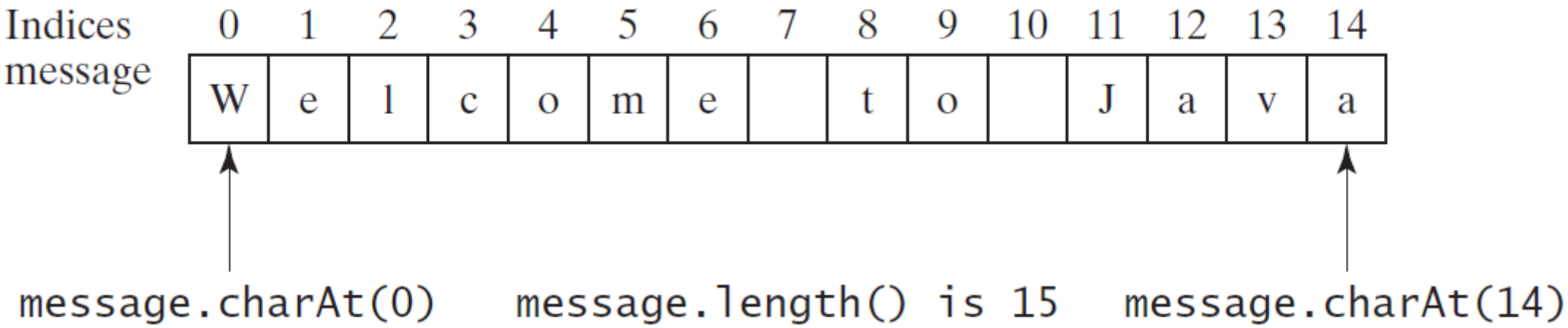
Simple Methods for **String** Objects

Method	Description
length()	Returns the number of characters in this string.
charAt(index)	Returns the character at the specified index from this string.
concat(s1)	Returns a new string that concatenates this string with string s1.
toUpperCase()	Returns a new string with all letters in uppercase.
toLowerCase()	Returns a new string with all letters in lowercase.
trim()	Returns a new string with whitespace characters trimmed on both sides.

Getting String Length

```
String message = "Welcome to Java";
System.out.println("The length of " + message + " is "
  + message.length());
```

Getting Characters from a String




```
String message = "Welcome to Java";
System.out.println("The first character in
message is " + message.charAt(0));
```

Converting Strings

```
"Welcome".toLowerCase() //returns a new string, welcome.

"Welcome".toUpperCase() //returns a new string, WELCOME.

"  Welcome  ".trim() //returns a new string, Welcome.
```

String Concatenation

```
String s3 = s1.concat(s2); or String s3 = s1 + s2; // Three strings s1, s2 concatenated and assigned to s3

String message = "Welcome " + "to " + "Java"; // Three strings are concatenated

String s = "Chapter" + 2; // String Chapter is concatenated with number 2, s becomes Chapter2

String s1 = "Supplement" + 'B'; // String Supplement is concatenated with character B s1 becomes SupplementB
```

Comparing Strings

Method	Description
`equals(s1)`	Returns true if this string is equal to string `s1`.
`equalsIgnoreCase(s1)`	Returns true if this string is equal to string `s1`; it is case insensitive.

Examples

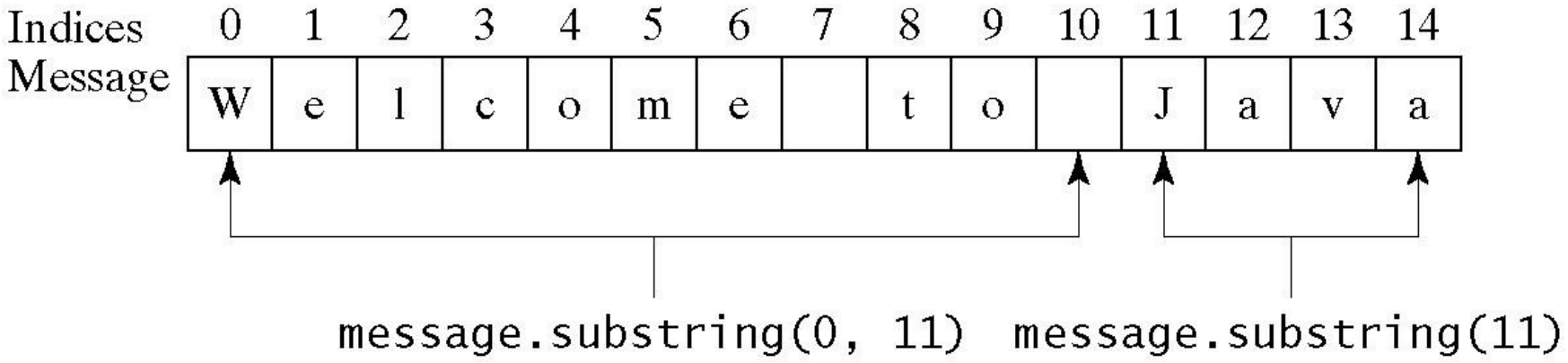
```
String s1 = "Welcome to Java";
String s2 = new String("Welcome to Java");
String s3 = "Welcome to Java";
String s4 = new String("welcome to JAVA");
System.out.println("s1 == s2 is " + (s1 == s2));
System.out.println("s1 == s3 is " + (s1 == s3));
System.out.println("s1.equals(s2) is " + (s1.equals(s2)));
System.out.println("s1.equals(s3) is " + (s1.equals(s3)));
System.out.println("s1.equalsIgnoreCase(s4) is" + (s1.equalsIgnoreCase(s4))
```

Outputs :

s1 == s2 is **false**
s1 == s3 is **true**
s1.equals(s2) is **true**
s1.equals(s3) is **true**
s1.equalsIgnoreCase(s4) is **true**

Obtaining Substrings

Method	Description
`substring(beginIndex)`	Returns this string's substring that begins with the character at the specified `beginIndex` and extends to the end of the string, as shown in the Figure
`substring(beginIndex, endIndex)`	Returns this string's substring that begins at the specified `beginIndex` and extends to the character at index `endIndex - 1`, as shown in the Figure Note that the character at `endIndex` is not part of the substring.

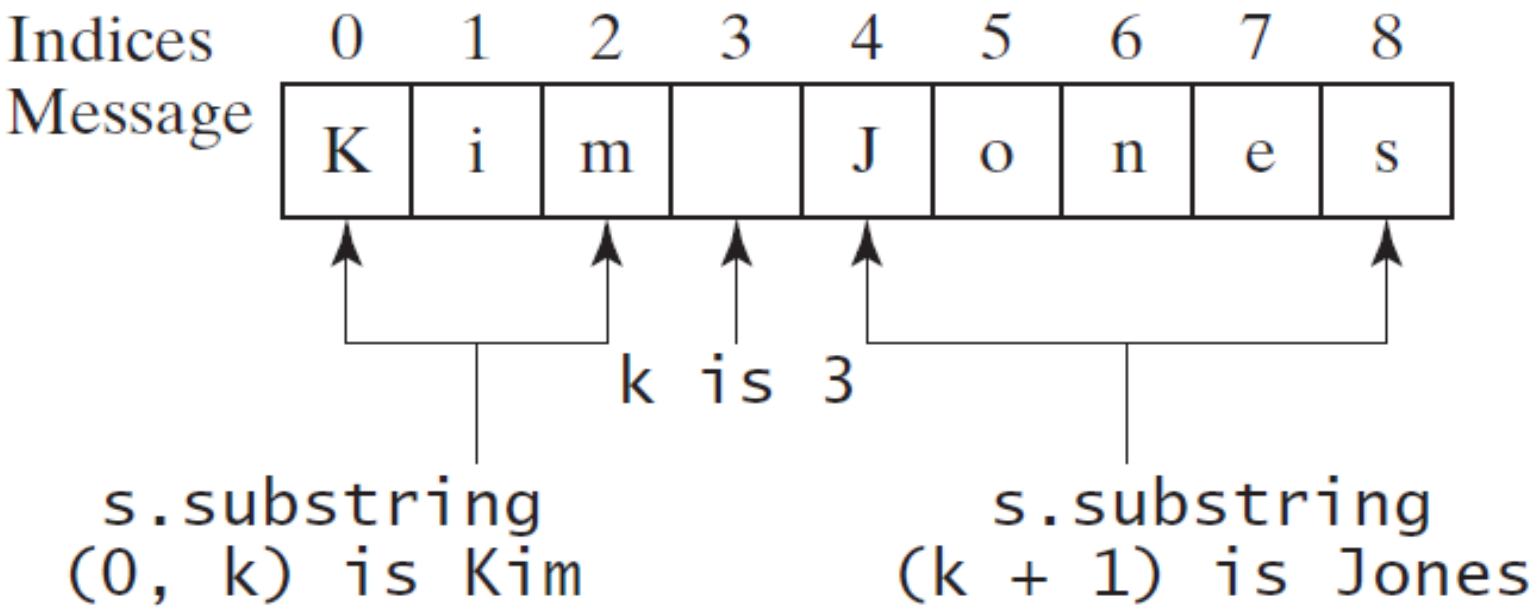


Finding a Character or a Substring in a String

Method	Description
`indexOf(ch)`	Returns the index of the first occurrence of `ch` in the string. Returns `-1` if not matched.
`indexOf(ch, fromIndex)`	Returns the index of the first occurrence of `ch` after `fromIndex` in the string. Returns `-1` if not matched.
`indexOf(s)`	Returns the index of the first occurrence of string `s` in this string. Returns `-1` if not matched.
`indexOf(s, fromIndex)`	Returns the index of the first occurrence of string `s` in this string after `fromIndex`. Returns `-1` if not matched.
`lastIndexOf(ch)`	Returns the index of the last occurrence of `ch` in the string. Returns `-1` if not matched.
`lastIndexOf(ch, fromIndex)`	Returns the index of the last occurrence of `ch` before `fromIndex` in this string. Returns `-1` if not matched.
`lastIndexOf(s)`	Returns the index of the last occurrence of string `s`. Returns `-1` if not matched.
`lastIndexOf(s, fromIndex)`	Returns the index of the last occurrence of string `s` before `fromIndex`. Returns `-1` if not matched.

Finding a Character or a Substring in a String

```
int k = s.indexOf(' ');
String firstName = s.substring(0, k);
String lastName = s.substring(k + 1);
```



The Math Class

- The **Math** class provides a number of standard mathematical methods
  - It is found in the **java.lang** package, so it does not require an **import** statement
- Class constants:
  - **PI** = 3.141592653589793
  - **E** = 2.718281828459045
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - **min**, **max**, **abs**, and **random** Methods

Exponent Methods

- **pow(**double a, double b**)**
  //Returns a raised to the power of b.
- **sqrt(**double a**)**
  //Returns the square root of a.

Examples:

```
Math.pow(2, 3);// returns 8.0
Math.pow(3, 2);// returns 9.0
Math.pow(3.5, 2.5);// returns 22.91765
Math.sqrt(4);// returns 2.0
Math.sqrt(10.5);// returns 3.24
```

min, max, and abs

`max(a, b)` and **`min(a, b)`**
//Returns the maximum or minimum of two parameters.

`abs(a)` //Returns the absolute value of the parameter.

`random()`
//Returns a random ***`double`*** value
//in the **range [0.0, 1.0)**.

Examples:

```
Math.max(2, 3); //returns 3
Math.max(2.5, 3); //returns 3.0
Math.min(2.5, 3.6); //returns 2.5
Math.abs(-2); //returns 2
Math.abs(-2.1); //returns 2.1
```

The random Method

- Generates a **random *double*** value **greater than or equal to 0.0 and less than 1.0** (0 <= Math.random() < 1.0).

```
(int)(Math.random() * 10)
```

→ Returns a random integer between 0 and 9.

```
50 + (int)(Math.random() * 50)
```

→ Returns a random integer between 50 and 99.

```
a + Math.random() * b
```

→ Returns a random number between a and a + b, excluding a + b.

Examples on random Method

For example:

```
int x = (50 + Math.random( )*50);
```
//returns a random number between 50 and 100, exclude (50 and 100) and store it in **x**

```
System.out.println(0 + Math.random( )*10);
```
// returns a random number between 0 and 10 exclude (0 and 10) and print it

```
double d = 50 + (int) (Math.random( )*50);
```
// returns a random integer number starting from 50 to 99 (50 and 99 inclusive) and stores it in **d**

Exercises

- Q: Write a java program to solve the following equations:

$$x = \sqrt{a^2 + b^2} + \frac{\sqrt{a + 3b - ab}}{|b - 5a^3|}$$

a=3 , b=1.5, x is integer.

- Q: Find the maximum number of 35, 10 , 62 .
- Q: Write a java program that declares a String variable message="Welcome to Java", and generates an integer random number between 0 and length of that String. Then prints the character at that random number.

Exercises

- Q: write a program to print a user name, if the name consists of more than 7 characters.
- Q: write a program to compute the area of a circle, if the radius is more than or equals to 2.5 where the radius should be entered as string ?
- Q: write a program to print “your name contains letter R” if the user name contains letter R.
- Q: Write a program to prompt a user to enter his name, and print "your name is too short" if the user name is less than or equals 5 characters, otherwise print "your name has more than 5 characters" , using SWITCH.

Exercises

- Q: Write a Java program to compare two strings lexicographically, ignoring case differences.

Sample Output:

String 1: This is exercise 1

String 2: This is Exercise 1

"This is exercise 1" is equal to "This is Exercise 1"

Exercises

- Write a Java program to test if a given string contains the specified sequence of *char* values.

Sample Output:

Original String: PHP Exercises and Python Exercises Specified sequence of char values: “and”

true

User Defined Methods

- **User-Defined Methods** are methods that are created by the user based on some requirements.
- Benefits of Methods:
  - Break a problem down into small manageable pieces. This is called **divide** and **conquer**.
  - Write a method once and reuse it anywhere.
  - Information hiding. Hide the implementation from the user.
  - Reduce complexity.
  - A method in Java is equivalent to a function, procedure, or subroutine in other languages.

Defining Methods

- A **method** is a collection of statements that are grouped together to perform an operation.

Define a method

```
public static int max(int num1, int num2) {

   int result;

   if (num1 > num2)
     result = num1;
   else
     result = num2;

   return result;
}
```

Invoke a method

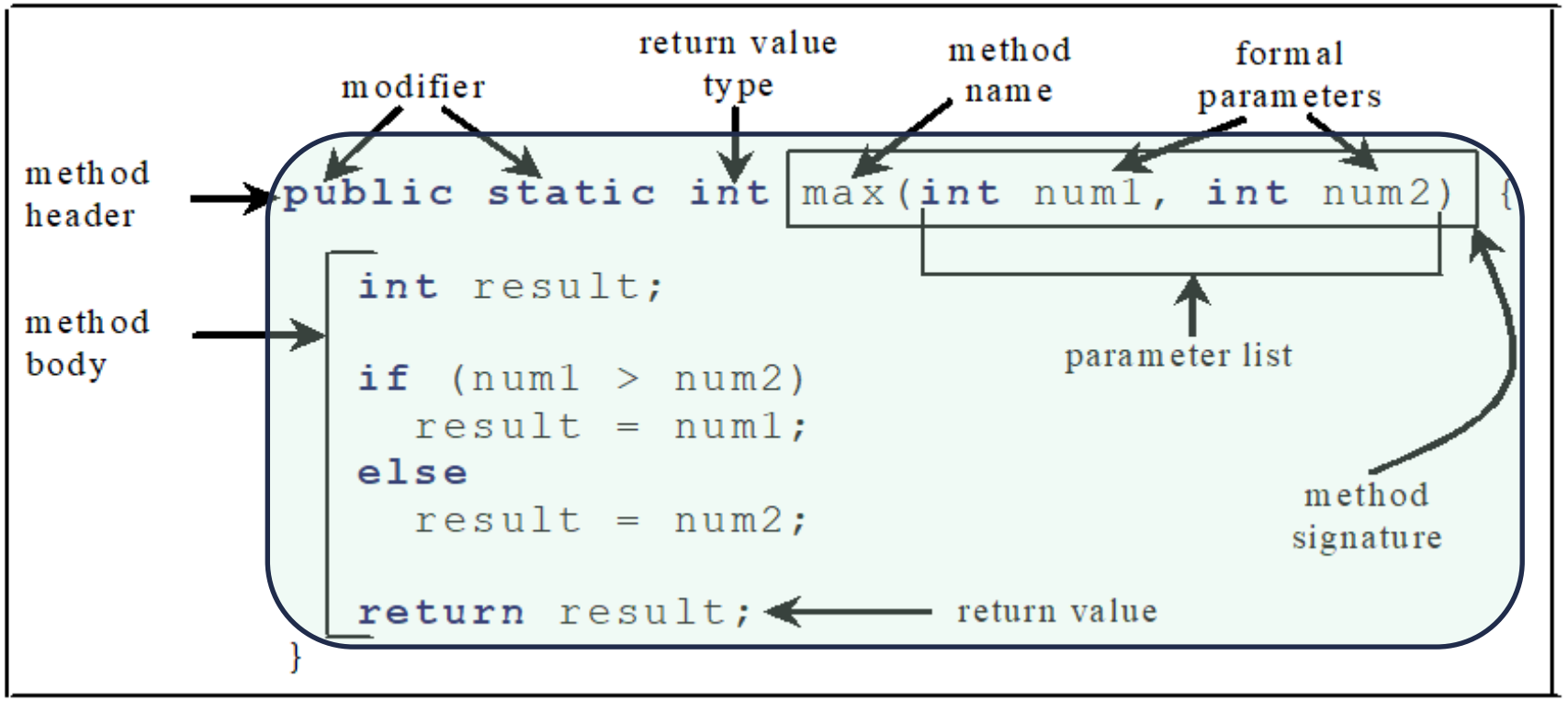
```
int z = max(x, y);
```

actual parameters
(arguments)

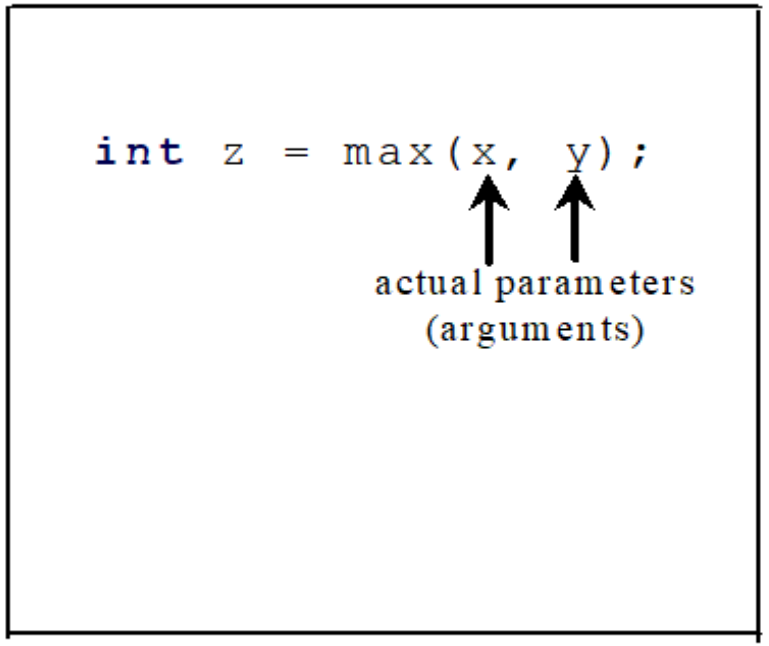
Defining Methods

- A **method** is a collection of statements that are grouped together to perform an operation.

Define a method



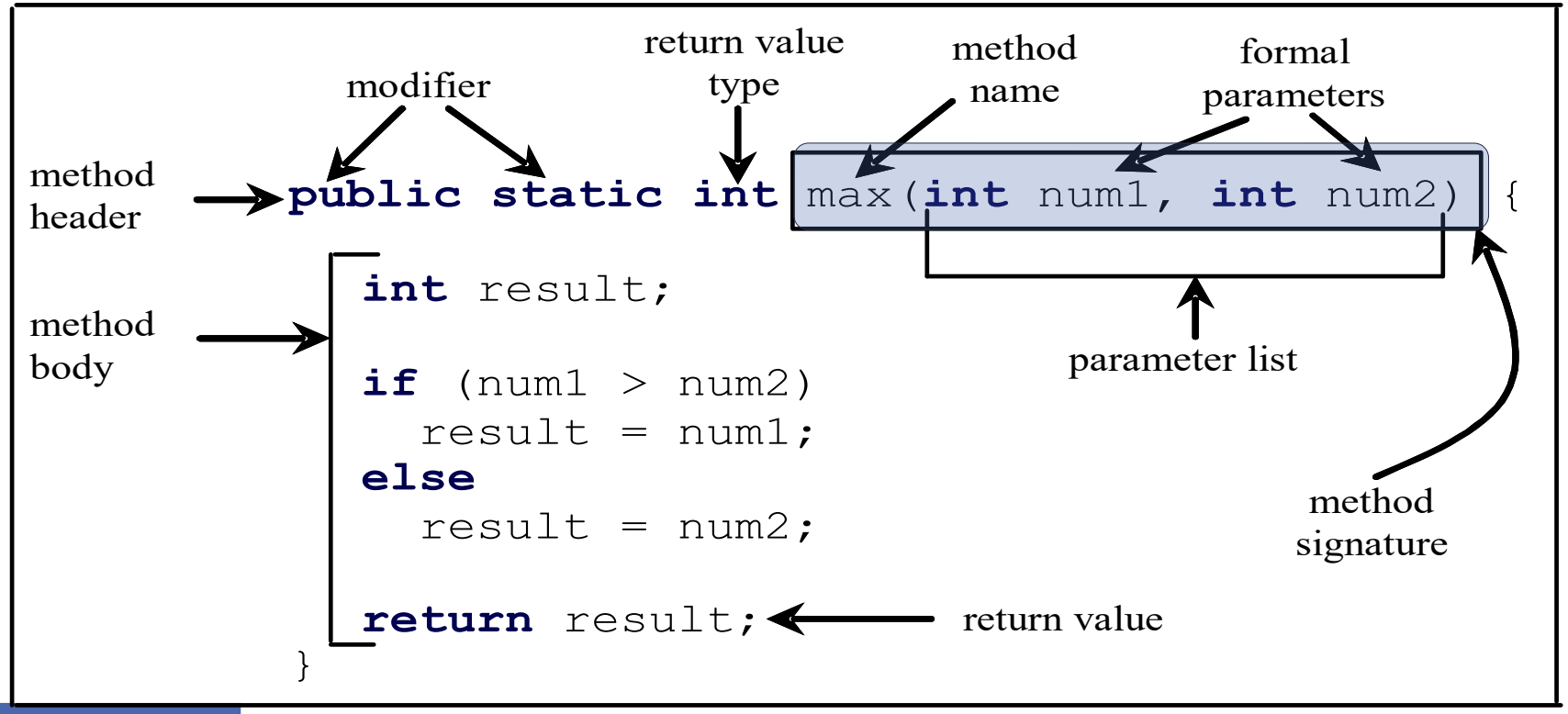
Invoke a method



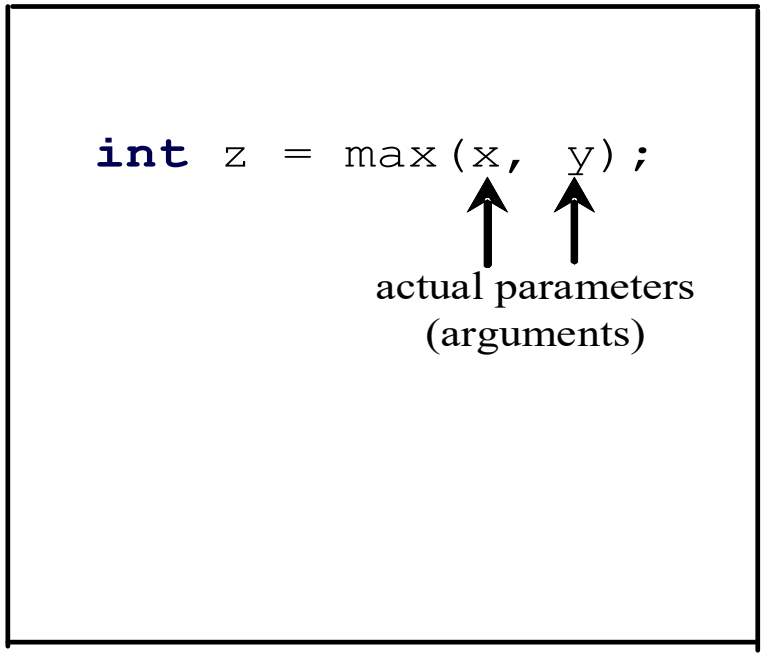
Method Signature

- **Method signature** is the combination of the method name and the parameter list.

Define a method



Invoke a method



Formal Parameters

- The variables defined in the method header are known as **formal parameters**.

Define a method

```
public static int max(int num1, int num2) {
    int result;

    if (num1 > num2)
      result = num1;
    else
      result = num2;

    return result;
}
```

method header

method body

modifier

return value type

method name

formal parameters

parameter list

method signature

return value

Invoke a method

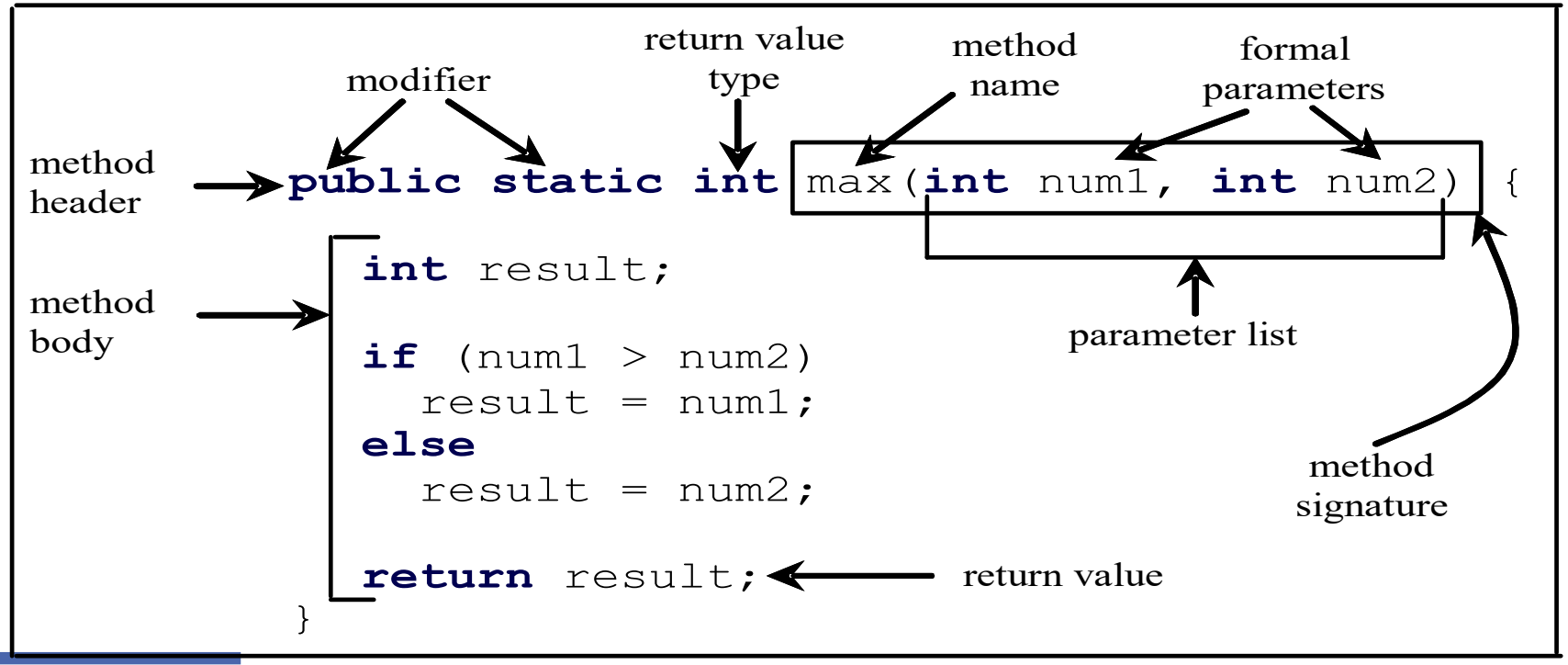
```
int z = max(x, y);
```

actual parameters (arguments)

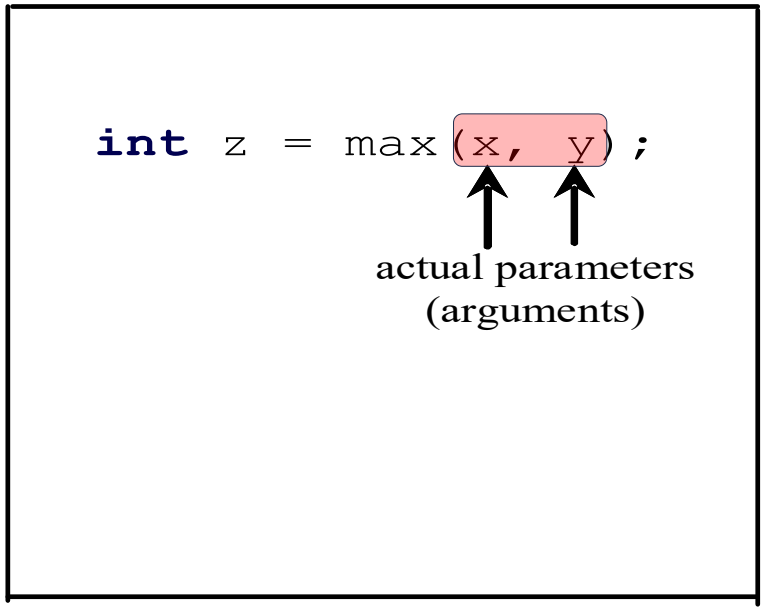
Actual Parameters

- When a method is invoked, you pass a value to the parameter. This value is referred to as ***actual parameter*** or **argument**.

Define a method



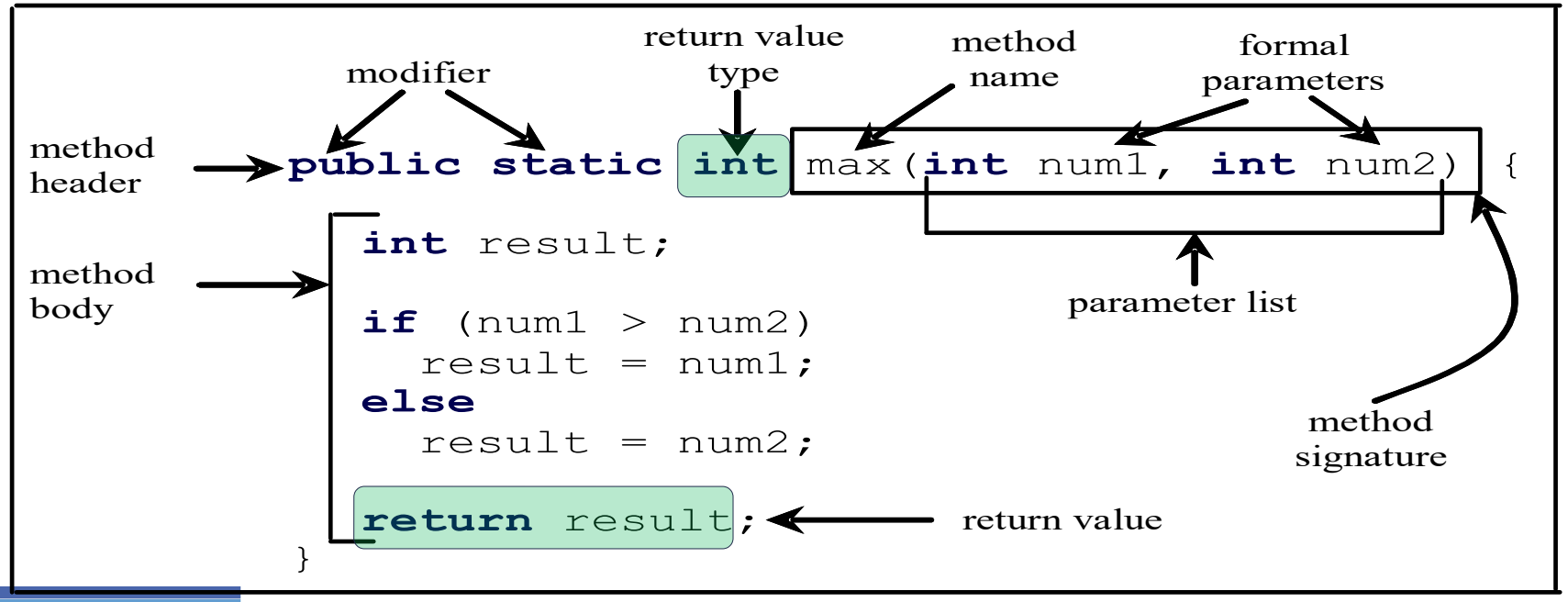
Invoke a method



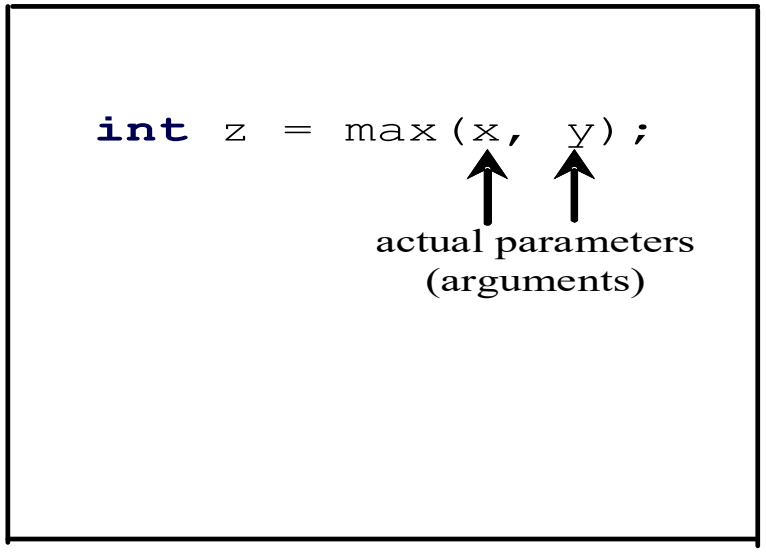
Return Value Type

- A method may return a value. The **returnValueType** is the data type of the value the method returns. If the method does not return a value, the **returnValueType** is the keyword **void**. For example, the **returnValueType** in the **main** method is **void**.

Define a method



Invoke a method



Code Tracing(10 Slides)

i is now 5

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

j is now 2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

invoke max(i, j)

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

invoke max(i, j)
Pass the value of i to num1
Pass the value of j to num2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

declare variable result

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

(num1 > num2) is true since num1 is 5 and num2 is 2

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

Result is now 5

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

return max(i, j) and assign the return value to k

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Code Tracing(10 Slides)

Execute the print statement

```
public static void main(String[] args) {
  int i = 5;
  int j = 2;
  int k = max(i, j);

  System.out.println(
   "The maximum between " + i +
   " and " + j + " is " + k);
}
```

```
public static int max(int num1, int num2) {
  int result;

  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

CAUTION

- A return statement is required for a value-returning method. The method shown below in (a) is logically correct, but it has a compilation error because the Java compiler thinks it possible that this method does not return any value.

```java
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else if (n < 0)
    return -1;
}
```

(a)

Should be →

```java
public static int sign(int n) {
  if (n > 0)
    return 1;
  else if (n == 0)
    return 0;
  else
    return -1;
}
```

(b)

Value-returning Methods

- A value-returning method is one that simply performs a task and then sends a value back to the code that called it.
- Used in expressions.
- Can save value for later calculation or print value.

```
public static double min(double x, double y)
  {   if (x <= y)
          return x;

      return y;    }
```

void Methods

- A **void** method is one that simply performs a task and then terminates.
- Similar in structure to value-returning methods.
- Call to method is always stand-alone statement.
- Can use **return** statement to exit method early.

```
public static void displayValue(int num)
{
   System.out.println("The value is " + num);
}
```

Passing Arguments by value

- In Java, all arguments of the **primitive data types** are ***passed by value***, which means that only a copy of an argument's value is passed into a parameter variable.
- A method's parameter variables are separate and distinct from the arguments that are listed inside the parentheses of a method call.
- If a parameter variable is changed inside a method, it has no affect on the original argument.

PassByValue.java

Passing Arguments by value

The value 5.0 of argument n1 is copied into the **num1** parameter.

The value 10.0 of argument n2 is copied into the **num2** parameter.

NOTE: Order matters!

```
double n1=5.0, n2=10.0
showSum(n1, n2);
```

```
public static void showSum(double num1, double num2)
{
   double sum;   //to hold the sum
   sum = num1 + num2;
   System.out.println("The sum is " + sum);
}
```

The static key word

- If a method is declared **static**, it can be accessed from the class itself without a need to create an object of that class.

Exercise(1)

- Write a Java method to find the **smallest** number among three numbers.
- Write a Java program that includes a method named **isOddEven** to determine whether a given integer is odd or even. Prompt the user to enter a whole number, call the **isOddEven** method to check its parity, and then print a message indicating whether the entered number is odd or even. Ensure that your program provides clear user instructions and outputs a meaningful result based on the parity of the entered number.

Exercise(2)

- write a program to find the area of a rectangle, as following:
  - Define a method to **read** the width and height of a rectangle.
  - Define a method to **calculate the area**.
  - Define a method to **print** the area.

Exercise(3)

- Write a program to find the factorial of a number using two methods: one method called **input** to get the input from the user and return it as integer, and the second called **fact** which computes and prints the factorial.
- Write a java method that prompts the user to input a positive integer. It should then print the **multiplication table** of that number.

Exercise(4)

- Write a java method to find the value of one number raised to the power of another **(x^y)**.
- Write a program to find the average of three marks for a student, as following:
  - Define a method named **getMark** to read the value of each mark, and returns it as int.
  - Define a method named **findAvg** to compute the average for the student.

Exercise(5)

- Write a java method named findVol to find the volume the volume of a cylinder, where: **cylinderVolume=πr²h**, Where **π** is constant and the values of **h** and **r** should be received as a parameters
- Write a java method called display() to print the even numbers in a range between x and y. where x and y should be entered by user.

Exercise(6)

- Write Java program that prompts the user to input two integers, a starting number and an ending number. It then uses a while loop to print all the natural numbers from the starting number to the ending number, inclusive.

Reference

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos : YouTube channel.
- Lab manual 2,3,4,7